EPIC 05 – SERVICE COMMUNICATION & RESILIENCE

Task 01: REST Client Communication

Task: Implement service communication using a modern Spring HTTP client approach.

For Spring Boot 3.x / Spring 6.x, use **RestClient**. It is the modern synchronous HTTP client and is simpler than the older RestTemplate.

Step 1 — Add dependency

In your service's pom.xml, make sure you have:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Save the file and allow Maven to update the project.

Step 2 — Create a REST client

Suppose your **Order Service** needs to communicate with the **Product Service**.

Create:

```
src/main/java
    └── com.example.orderservice
        └── client
            └── ProductClient.java
```

Add,

```
package com.example.orderservice.client;

import org.springframework.stereotype.Component;
import org.springframework.web.client.RestClient;

@Component
public class ProductClient {

   private final RestClient restClient;

   public ProductClient(RestClient.Builder builder) {
      this.restClient = builder
            .baseUrl("http://localhost:8082")
            .build();
   }

   public ProductResponse getProductById(Long productId) {

      return restClient.get()
            .uri("/products/{id}", productId)
            .retrieve()
            .body(ProductResponse.class);
   }
}
```

Step 3 — Create ProductResponse

Create:

client/ProductResponse.java

Code:

```
package com.example.orderservice.client;

public class ProductResponse {
```

```
    private Long id;
    private String name;
    private Double price;

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public Double getPrice() {
        return price;
    }

    public void setPrice(Double price) {
        this.price = price;
    }
}
```

Step 4 — Use the client from your service

Example:

```
@Service
public class OrderService {
```

```
    private final ProductClient productClient;

    public OrderService(ProductClient productClient) {
        this.productClient = productClient;
    }

    public ProductResponse getProduct(Long productId) {
        return productClient.getProductById(productId);
    }
}
```

If you don't already have @Service imported:

```
import org.springframework.stereotype.Service;
```

Step 5 — Create an endpoint to test it

In your Order Controller:

```
@RestController
@RequestMapping("/orders")
public class OrderController {

    private final OrderService orderService;

    public OrderController(OrderService orderService) {
        this.orderService = orderService;
    }

    @GetMapping("/product/{id}")
    public ProductResponse getProduct(@PathVariable Long id) {
        return orderService.getProduct(id);
    }
}
```

Imports:

import org.springframework.web.bind.annotation.GetMapping;

import org.springframework.web.bind.annotation.PathVariable;

import org.springframework.web.bind.annotation.RequestMapping;

import org.springframework.web.bind.annotation.RestController;

Step 6 — How communication works

Client

|

| GET /orders/product/1

v

Order Service

|

| RestClient

| GET http://localhost:8082/products/1

v

Product Service

|

| Product details

v

Order Service

|

v

Client

Step 7 — Run and test

Start:

Product Service → port 8082
Order Service → port 8081

Then test:

GET http://localhost:8081/orders/product/1

The **Order Service** uses RestClient to call the **Product Service**.

Task 01 is completed when

- Product Service is running.
- Order Service is running.
- RestClient is configured.
- Order Service successfully calls Product Service.
- Product data is returned through the Order Service endpoint.